

UNIT - 1

Program

Program

A Program is a set of statements.

A set of coded instructions that a computer can understand to solve a problem or produce a desired result. Two basic types of computer programs are

(1) an operating system, which provides the most fundamental instructions a computer uses in its operations, and

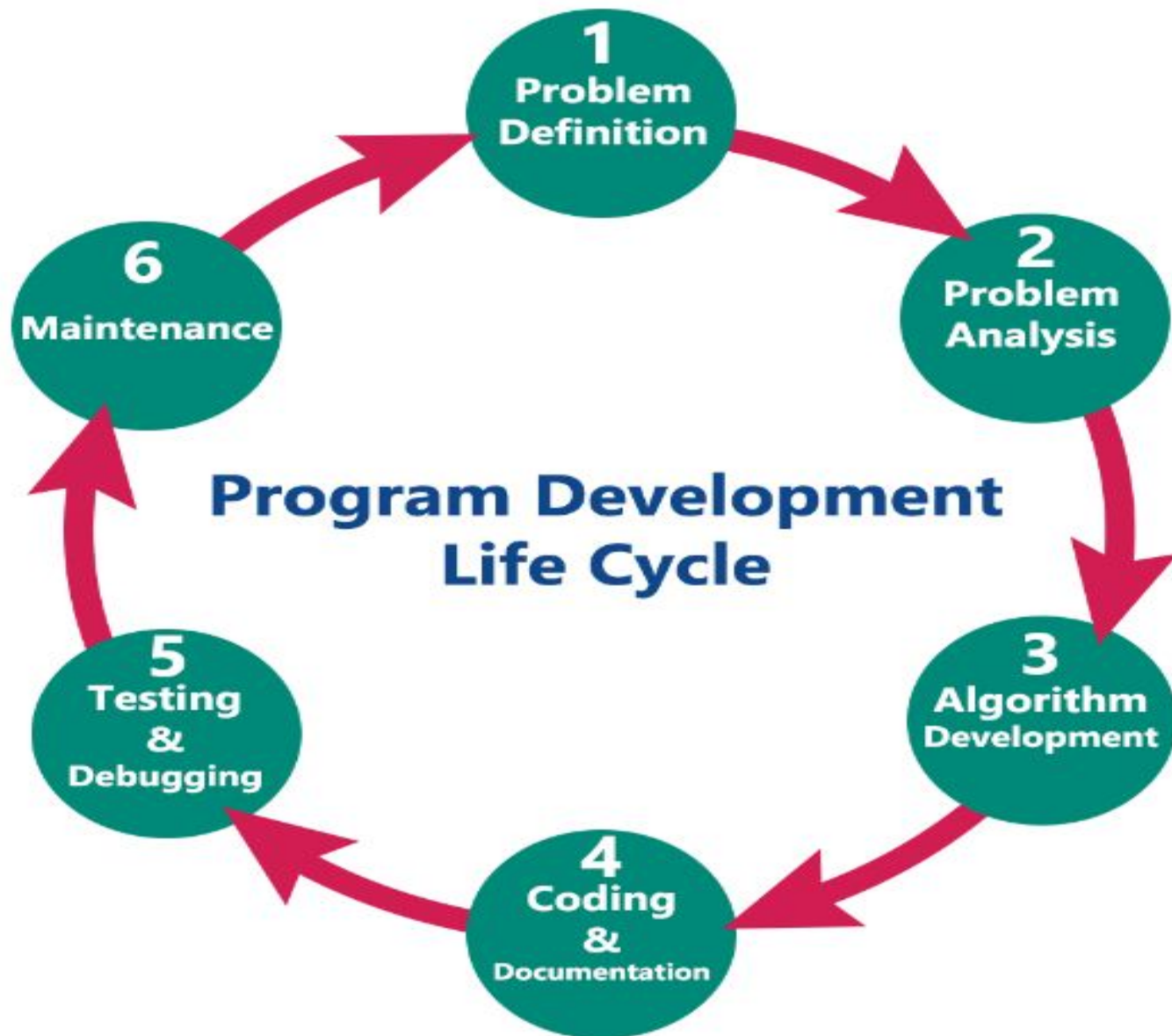
(2) an application program, which runs on the operating system and does a specific job.

Program Development Cycle

When we want to develop a program using any programming language, we follow a sequence of steps. These steps are called phases in program development. The program development life cycle is a set of steps or phases that are used to develop a program in any programming language.

Generally, program development life cycle contains 6 phases, they are as follows....

- Problem Definition**
- Problem Analysis**
- Algorithm Development**
- Coding & Documentation**
- Testing & Debugging**
- Maintenance**



Program Development Cycle

1. Problem Definition

In this phase, we define the problem statement and we decide the boundaries of the problem. In this phase we need to understand the problem statement, what is our requirement, what should be the output of the problem solution. These are defined in this first phase of the program development life cycle.

2. Problem Analysis

In phase 2, we determine the requirements like variables, functions, etc. to solve the problem. That means we gather the required resources to solve the problem defined in the problem definition phase.

Program Development Cycle

3. Algorithm Development

During this phase, we develop a step by step procedure to solve the problem using the specification given in the previous phase. This phase is very important for program development. That means we write the solution in step by step statements.

4. Coding & Documentation

This phase uses a programming language to write or implement actual programming instructions for the steps defined in the previous phase. In this phase, we construct actual program. That means we write the program to solve the given problem using programming languages like C, C++, Java etc.,

Program Development Cycle

5. Testing & Debugging

During this phase, we check whether the code written in previous step is solving the specified problem or not. That means we test the program whether it is solving the problem for various input data values or not. We also test that whether it is providing the desired output or not.

6. Maintenance

During this phase, the program is actively used by the users. If any enhancements found in this phase, all the phases are to be repeated again to make the enhancements.

PROGRAMMING LANGUAGE

Programming languages are used to create programs. The Basic purpose of Programming Language is to develop software which ultimately solves a given problem.

Such languages includes BASIC, C, C++, COBOL, FORTRAN, Ada, and Pascal ,etc.

While programming languages are easy for the humans to read and understand, the computer understands the machine language that consists of numbers only that is binary language.

GENERATION OF PROGRAMMING LANGUAGES

The concept of generations of programming languages also known as “Levels” is closely connected to the advances in technology that brought about the different generations of computers.

There are five generations of Programming languages.

FIRST GENERATION: MACHINE LANGUAGE

Machine (or low level) language was used to program the first stored-program computer systems. It is the only language that the computer understands.

All the commands and data values are expressed using 1s and 0s.

In 1950s each computer had its own native language, and programmers had to combine numbers to represent instructions such as *add* and *subtract*.

FIRST GENERATION: MACHINE LANGUAGE

Although there were similarities between each of the machine language but a computer could not understand programs written in another machine language.

The main advantage of machine language is that the code can run very fast and efficiently, since it is directly executed by the CPU.

However, this language is difficult to learn and is far more difficult to edit if errors occur.

Also code written in machine language is not portable.

SECOND GENERATION: ASSEMBLY LANGUAGE

The second generation includes the assembly language which is a symbolic language that use symbols to represent machine-language instructions.

These languages are closely connected to machine language and the internal architecture of the computer system on which they are used.

An assembly language statement consists of a label and an operation code.

SECOND GENERATION: ASSEMBLY LANGUAGE

However, like the machine language, assembly language is also machine-dependent.

Programs written in assembly language need a translator often known as the assembler to convert them into machine language.

THIRD GENERATION : HIGH LEVEL LANGUAGE

In these languages, the program statements are not closely related to the internal characteristics of the computer and is therefore often referred to as high-level languages.

3GLs made programming easier, efficient and less prone to errors. 3GL includes languages like FORTRAN and COBOL and C.

THIRD GENERATION : HIGH LEVEL LANGUAGE

A translator is needed to translate the instructions written in high level language into computer-executable machine language. Such translators are commonly known as interpreters and compilers.

3GLs makes it easier to write and debug a program and gives the programmer more time to think about its overall logic. The programs written in such languages are portable between machines.

FOURTH GENERATION : VERY HIGH LEVEL LANGUAGE

4GLs are nonprocedural languages. When writing code using a procedural language, the programmer has to tell the computer how a task is done.

while using a nonprocedural language the programmers define only what they want the computer to do, without supplying all the details of how it has to be done.

FOURTH GENERATION : VERY HIGH LEVEL LANGUAGE

Characteristics of 4GLs include:

The code is written in English-like sentences. The code is easier to maintain.

4GL code enhances the productivity of the programmers as they have to type fewer lines of code to get something done.

FIFTH GENERATION PROGRAMMING LANGUAGE

5GLs are centered on solving problems using constraints given to the program, rather than using an algorithm written by a programmer.

They are widely used in artificial intelligence research. Typical examples of a 5GL include Prolog and Mercury.

Another aspect of a 5GL is that it contains visual tools to help develop a program.

Features of a good programming language

- The language must allow the programmer to write simple, clear and concise programs.
- The language must be simple to use so that a programmer can learn it without any explicit training.
- The language must be platform independent. That is, the program developed using the programming language can run on any computer system.
- The Graphical User Interface (GUI) of the language must be attractive, user-friendly, and self-explanatory.

Algorithm

“Algorithm” : It is a step by step descriptive procedure for solving a particular problem.

The algorithm gives logic of the program, that is, a step-by-step description of how to arrive at a solution.

Algorithm

A sequence of instructions must process the following characteristics:

- Instructions must be clear**
- Instructions must be effective.**
- Not even a single instruction must not be repeated infinitely**
- After the algorithm gets terminated, the desired result must be obtained**

Algorithm

Every algorithm always starts with START and terminate with STOP keywords.

In other way we can write algorithm using BEGIN and terminate with END keywords.

For input and output data we can use READ / INPUT for input and WRITE / PRINT for output.

For each Step We use Step-Number.

KEY FEARTURES OF AN ALGORITHM

Any algorithm has a finite number of steps and some steps may involve decision making, repetition. Broadly speaking, an algorithm having three key features that can be given as:

- 1. Sequence**
- 2. Decision**
- 3. Repetition**

KEY FEATRTURES OF AN ALGORITHM

1. Sequence

Sequence means that each step of the algorithm is executed in the specified order.

Ex. Input Two Number and Perform Addition

Step 1: Start

Step 2: Declare Variables A,B,Sum

Step 3: Input Value A

Step 4: Input Value B

Step 5: $\text{Sum} = \text{A} + \text{B}$

Step 6: Print Sum

Step 7: Stop

KEY FEARTURES OF AN ALGORITHM

2. Decision

Decision statements are used when the outcome of the process depends on some condition.

For Example, if $A > B$ then print “A is Greater” otherwise “B is Greater”

General Form of If as follows

if condition

then process 1

else

process 2

KEY FEARTURES OF AN ALGORITHM

Ex. Input Two Number and Find Maximum

Step 1 : Start

Step 2 : Declare A, B

Step 3 : Input Value A

Step 4 : Input Value B

Step 5 : if $A > B$ then

Print "A is Max"

Step 6 : else

Print "B is Max"

Step 7: Stop

KEY FEARTURES OF AN ALGORITHM

3. Repetition

Repetition which involves executing one or more steps for a number of times can be implemented using constructs like the while, do-while and for loops.

These loops executed one or more steps until some condition is true.

KEY FEARTURES OF AN ALGORITHM

Ex. Print 1 to 10 Numbers using Loop.

Step 1: Start

Step 2: [Initialize] set $I=1$, $N=10$

Step 3: Repeat Step 3 While $I \leq N$ Otherwise Go to Step-7

Step 4: Print I

Step 5 : $I=I+1$

Step 6: Go to Step-3

Step 7: Stop

ADVANTAGE OF ALGORITHM

A programmer can know identification of the processes, major decision statements and required variables for solving the problem at the development time.

Algorithm represents a process of an entire solution in a specific sequence so it is better way to understand program.

FLOWCHART

A flow chart is a graphical or symbolic representation of a process.

They are basically used to design and document complex processes to help the viewers to visualize the logic of the process

Each step in the process is represented by different shapes and displays a short detail of the process step. Every symbol is connected together with arrows showing the process flow direction.

FLOWCHART

Various Symbols of Flowchart is as Follows:

1. Start and end symbols



It is also known as the terminal symbol is always the first and the last symbol in a flowchart.

FLOWCHART

2. Input/Output Symbols



This *Symbol* is used to get input from users or display the results to them

FLOWCHART

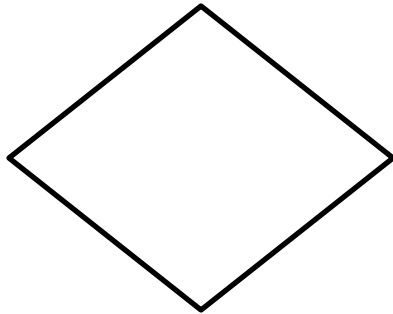
3. Generic processing



Generic processing step also called an activity is represents instructions like add a to b, save the result.

FLOWCHART

4. Conditional or decision symbol



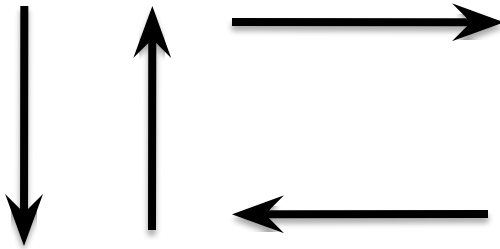
It is represented using a diamond.

It is used to depict a Yes/No question or True/False test.

The Arrow should always be labeled.

FLOWCHART

5. Arrow

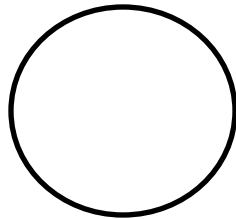


Arrows depict the flow of control of the program.

They illustrate the exact sequence in which the instructions are executed.

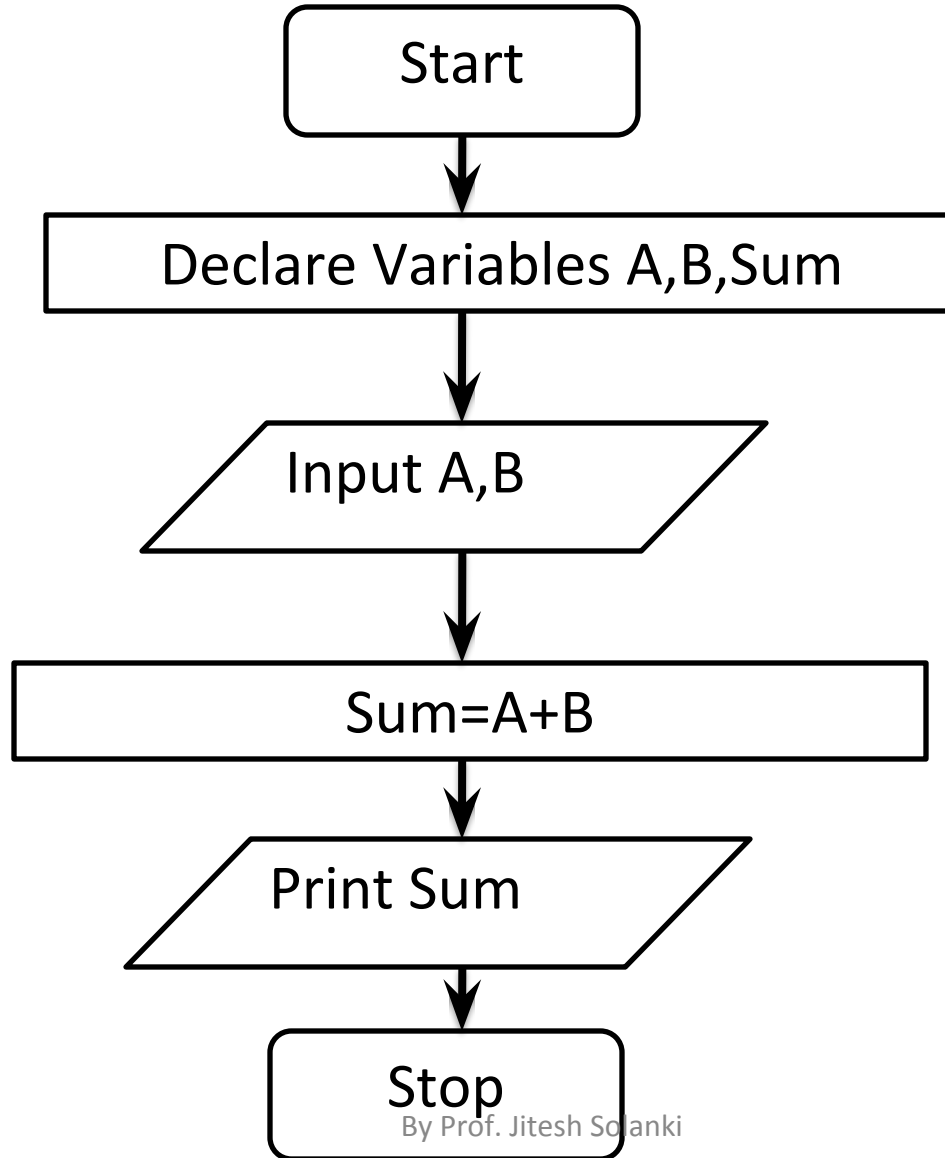
FLOWCHART

6. Labeled connectors

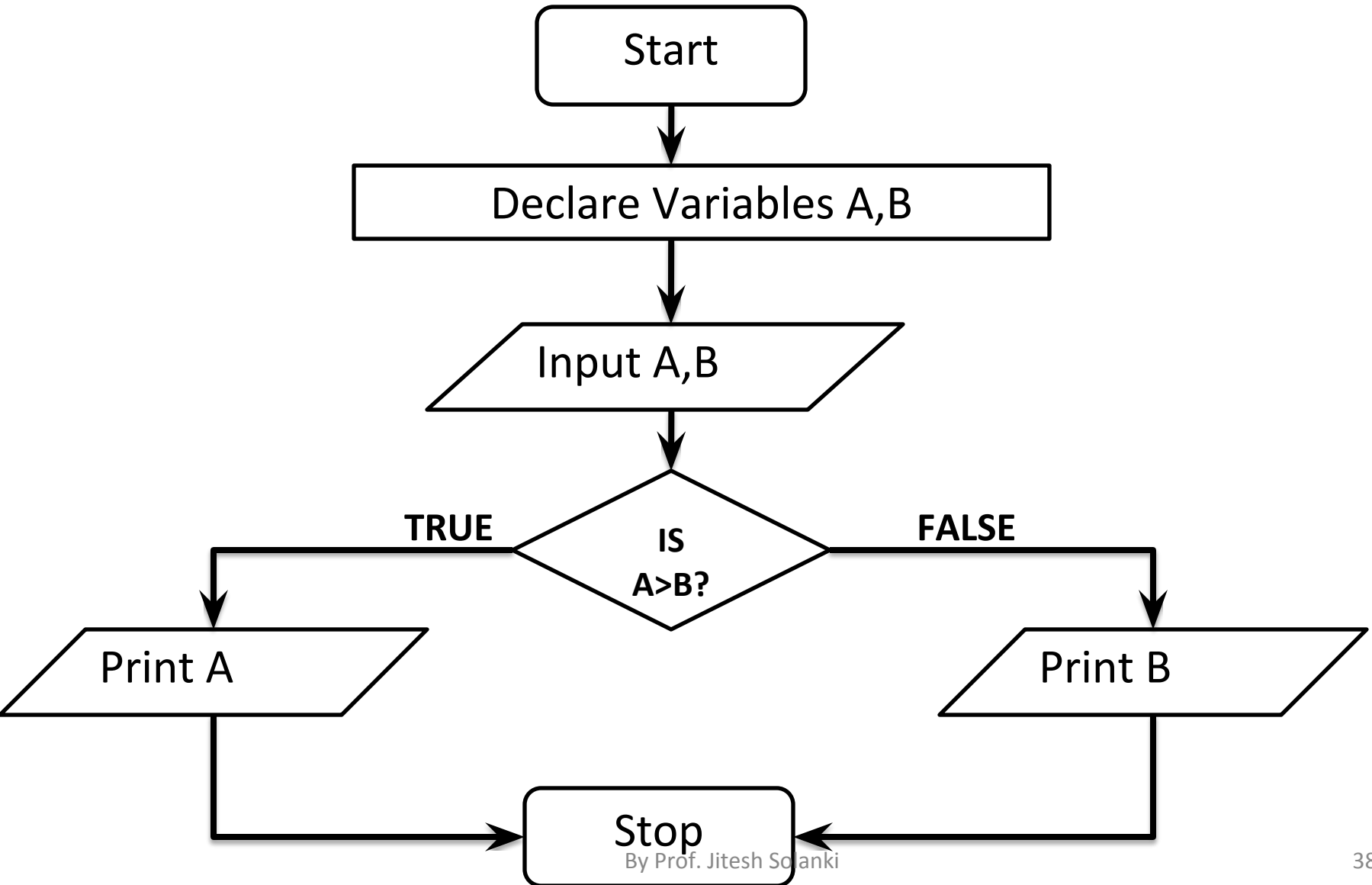


connectors represented by an identifying label inside a circle are used in complex or multi-sheet diagrams to substitute for arrows.

Example of Flowchart



Example of Flowchart



Advantages of Flowcharts

A flowchart is a diagrammatic representation that illustrates the sequence of steps that must be performed to solve a problem.

They are usually drawn in the early stages of formulating computer solutions to facilitate communication between programmers and business people.

Flowcharts help programmers to understand the logic of complicated and lengthy problems.

They help to analyze the problem in a more effective manner

Limitations of using Flowcharts

Drawing flowcharts is a laborious and a time consuming activity.

Flowchart of a complex program becomes complex. At times, a little bit of alteration in the solution may require complete re-drawing of the flowchart.

THANK YOU